



G.R.D. Ground Rescue Drone

1/12/2026

Francisco Javier Aguayo Hernández

University of Málaga

Index

Index	1
Introduction	2
What advantages does this model offer?	2
Hardware	3
Control Platform: Microcontrollers	3
ESP32-CAM	3
ESP32	3
Drive System	3
Two DC motors, 120 rpm, 12V nominal	3
Two 80mm RC wheels	3
L298N module	3
Power System	4
Rechargeable 9V battery	4
Connectivity	4
MM, MF, FF cables	4
WiFi network 802.11 b/g/n	4
Mechanical Structure	4
3D-printed chassis (Ender 3 v3)	4
Simplified Functional Diagram	5
Software	6
Motor Control & PS4 Input	6
Camera Streaming & Face Detection	10
MainCam_Program	10
Camera_Pins	12
Análisis de Costes	12
Component	13
Development Costs (non-recurring)	14
Market Comparison	14
Schematics and Models	15
Electrical Wiring Diagram (Fritzing)	15
Chassis Design (SolidWorks)	15
LED Bar Design (SolidWorks)	16
Motor Mount Design (SolidWorks)	16
Full Assembly (SolidWorks)	17
ESP32-CAM Live View	17

Introduction

This project develops a low-cost ground drone based on an ESP32 microcontroller, capable of moving via teleoperation and sending video from an ESP32-CAM camera mounted on board. The system is aimed at military or rescue applications.

There are currently numerous robots and microcontroller-driven cars on the market (mBot, Arduino cars, budget 2WD platforms, etc.), but most focus on basic movement functions or short-range Bluetooth control. The proposed development improves on these solutions by integrating WiFi connectivity, wheel encoder readings to measure speed, and a camera that allows real-time environmental monitoring, all while keeping costs low thanks to the use of widely available hardware.

The ground drone is conceived as an open and extensible platform: beyond basic movement functions, it can be expanded with navigation algorithms, autonomous navigation, AI integration, control via a web interface or mobile app, remote image processing on a PC, and additional sensors.

What advantages does this model offer?

The compact size of the cylindrical design (approximately 10 cm in length and 80 mm wheel diameter) makes it ideal for operating in confined spaces, rubble, tunnels, or collapsed structures where larger robots cannot access, improving effectiveness in search and rescue operations for trapped people.

The reduced dimensions and low profile make the platform difficult to detect in military tactical environments, providing a stealth advantage over enemy defense systems or in reconnaissance missions where discretion is critical. Finally, being entirely manufactured through 3D printing with the Ender 3 v3, the chassis is fully customizable: camouflage paint can be applied to adapt to different operational theatres, something impossible on commercial platforms of similar cost, which broadens the versatility of the platform without a significant increase in costs.

Hardware

The remote vision robot is built on a modular architecture that integrates microcontrollers, actuators, vision systems, and a 3D-printed structure. Below are the key components:

Control Platform: Microcontrollers

ESP32-CAM

Microcontroller module with integrated camera (OV2640 sensor, resolution up to 1600×1200 pixels) and WiFi 802.11 b/g/n connectivity. This is the central component that captures video, runs the web server for live streaming, and partially controls the platform logic. Operates at 3.3V with typical consumption of 150–200 mA during video transmission, requiring stable, low-noise power supply.

ESP32

Secondary microcontroller dedicated to motor control and peripheral sensor management. Provides enough GPIO and PWM ports to drive both L298N modules independently, allowing differential speed control on each wheel.

Drive System

Two DC motors, 120 rpm, 12V nominal

Direct current motors with integrated gearbox. Although rated at 12V, they operate stably at 7.4V providing slow and controlled movement (< 0.5 m/s), ideal for tactical operations where precision takes priority over speed.

Two 80mm RC wheels

Rubber wheels with tyre tread pattern, providing traction on the irregular surfaces typical of rubble or rough terrain in rescue zones.

L298N module

Motor drivers that amplify low-voltage PWM signals (3.3V from ESP32) to motor voltages (7.4V). Each module controls one motor independently in both directions (forward, backward, braking). The modules operate with a typical internal voltage drop of 2V, delivering approximately 5.4V net to the motors from the 7.4V battery.

Power System

Rechargeable 9V battery

Provides power to the motors and, through a step-down regulator, is reduced to 5V to power the ESP32-CAM and logic modules. Estimated operating time: 10–15 minutes of continuous movement with active video streaming.

Connectivity

MM, MF, FF cables

Module interconnection allows reconfigurability and mechanical decoupling for servicing.

WiFi network 802.11 b/g/n

Built into the ESP32-CAM; transmits live video to web browsers on the operator's PC with typical latency of 200–500 ms depending on signal quality.

Mechanical Structure

3D-printed chassis (Ender 3 v3)

- Internally houses the battery, ESP32, and L298N module on a flat internal tray. Provides structural protection and characteristic compact shape.
- Lateral conical supports: DC motor housings with precise axis alignment; transmit torque to the wheels while keeping the centre of gravity low.
- Front opening: Access point for mounting the ESP32-CAM with direct forward view; allows video capture without obstructions.
- Top panel: Rectangular slot for mounting LED status indicator strip; visible from any operational angle.
- Rear hatch: Access to internal components; the handle acts as a balanced "antenna", providing a third support point for stability on inclined terrain or during emergency braking.

Simplified Functional Diagram

Component	Qty	Specification	Voltage	Function
ESP32-CAM	1	OV2640, WiFi 802.11 b/g/n, 3.3V logic	5V entrada → 3.3V interno	Video capture, web server, WiFi streaming
ESP32	1	32-bit, 240 MHz, 30+ GPIO, PWM	5V	Motor control, general logic
L298N module	1	Dual H-bridge, 2A per channel	7-12V entrada, 5V salida	PWM amplification, bidirectional motor control, 5V regulation
DC motor 120 rpm	2	12V nominal, reduction gearbox	5-9V nominal	Differential propulsion (left and right wheel)
RC wheel 80mm	2	Rubber, tyre tread pattern	N/A	Traction on irregular terrain
Rechargeable 9V battery	1	NiMH 200-250 mAh (rechargeable)	9V nominal	Main system power supply
MM/MF/FF cables	Varios	22 AWG dupont	N/A	Module interconnection
LEDs	5 unidades	5V	5V	Status indicators
PLA/PETG filament (Ender 3 v3)	~300-400 g	For 3D printing	N/A	

Software

The main system is programmed using Arduino IDE with the ESP32 framework. For mechanical design, SolidWorks is used, and for electrical schematics, Fritzing. This combination allows rapid firmware development, professional 3D modelling, and clear connection documentation.

Motor Control & PS4 Input

This module manages the differential traction of the ground drone via remote control using a PS4 gamepad. The ESP32 GPIO pins are defined for motor PWM (through the L298N) and wireless command reception. It includes wheel encoder reading for basic odometry and PID speed control, optimising movement on irregular terrain.

```
#include <Ticker.h>
#include <PS4Controller.h>

//Pins motor 1
const int ENCODER1_A = 26;
const int ENCODER1_B = 25;
const int MOTOR1_PIN_A = 23;
const int MOTOR1_PIN_B = 32;

//Pins motor 2
const int ENCODER2_A = 25;
const int ENCODER2_B = 26;
const int MOTOR2_PIN_A = 32;
const int MOTOR2_PIN_B = 33;

// PWM
const int freq = 1000;
const int resolution = 8;

// PID
const float KP = 1.0;
const float KI = 0.05;
const float KD = 0.02;
const float pidInterval = 0.01;

struct MotorPID {
    volatile long pos = 0;
    volatile int lastA = 0;
    volatile int lastB = 0;

    long target = 0;
    float lastError = 0;
    float integral = 0;
    int pwm = 0;
};

MotorPID m1, m2;
Ticker pidTimer;
```

```

// PS4 Controller
const char *ssid = "PS4-BLUETOOTH";
const char *password = "";

//ENCODERS
void IRAM_ATTR enc1A() {
    int A = digitalRead(ENCODER1_A);
    int B = digitalRead(ENCODER1_B);
    if (A != m1.lastA) {
        if (B != A) m1.pos++;
        else m1.pos--;
        m1.lastA = A;
    }
}

void IRAM_ATTR enc1B() {
    int A = digitalRead(ENCODER1_A);
    int B = digitalRead(ENCODER1_B);
    if (B != m1.lastB) {
        if (A == B) m1.pos++;
        else m1.pos--;
        m1.lastB = B;
    }
}

void IRAM_ATTR enc2A() {
    int A = digitalRead(ENCODER2_A);
    int B = digitalRead(ENCODER2_B);
    if (A != m2.lastA) {
        if (B != A) m2.pos++;
        else m2.pos--;
        m2.lastA = A;
    }
}

void IRAM_ATTR enc2B() {
    int A = digitalRead(ENCODER2_A);
    int B = digitalRead(ENCODER2_B);
    if (B != m2.lastB) {
        if (A == B) m2.pos++;
        else m2.pos--;
        m2.lastB = B;
    }
}

//PID programs
void updateMotorPID(MotorPID &m, int pinA, int pinB) {
    float error = m.target - m.pos;
    m.integral += error * pidInterval;
    m.integral = constrain(m.integral, -1000, 1000);
    float deriv = (error - m.lastError) / pidInterval;

    float out = KP * error + KI * m.integral + KD * deriv;
    int pwm = constrain((int)fabs(out), 0, 255);

    if (out > 0) {
        ledcWrite(pinA, pwm);
        ledcWrite(pinB, 0);
    } else if (out < 0) {

```



```

    ledcWrite(pinA, 0);
    ledcWrite(pinB, pwm);
} else {
    ledcWrite(pinA, 0);
    ledcWrite(pinB, 0);
}

m.pwm = pwm;
m.lastError = error;
}

void calcPID() {
    updateMotorPID(m1, MOTOR1_PIN_A, MOTOR1_PIN_B);
    updateMotorPID(m2, MOTOR2_PIN_A, MOTOR2_PIN_B);
}

void ps4Event(PS4EventType type, PS4Message &msg) {
    if (type == PS4_EVENT_BUTTON) {
        if (msg.button == PS4_BUTTON_CROSS) {
            m1.target = 0;
            m2.target = 0;
        }
    } else if (type == PS4_EVENT_ANALOG_STICK) {
        int16_t leftY = msg.analog.stick.ly;
        int16_t leftX = msg.analog.stick.lx;
        int16_t rightX = msg.analog.stick.rx;

        // Left stick: forward/backward
        long moveSpeed = map(leftY, -127, 127, -500, 500);

        // Left Joystick: Turn left/right
        long turnSpeed = map(rightX, -127, 127, -300, 300);

        // Differential: forward/backward with steering
        m1.target = moveSpeed + turnSpeed;
        m2.target = moveSpeed - turnSpeed;
    }
}

void setup() {
    Serial.begin(115200);

    // PWM
    ledcAttach(MOTOR1_PIN_A, freq, resolution);
    ledcAttach(MOTOR1_PIN_B, freq, resolution);
    ledcAttach(MOTOR2_PIN_A, freq, resolution);
    ledcAttach(MOTOR2_PIN_B, freq, resolution);

    // Encoders
    pinMode(ENCODER1_A, INPUT);
    pinMode(ENCODER1_B, INPUT);
    pinMode(ENCODER2_A, INPUT);
    pinMode(ENCODER2_B, INPUT);

    m1.lastA = digitalRead(ENCODER1_A);
    m1.lastB = digitalRead(ENCODER1_B);
    m2.lastA = digitalRead(ENCODER2_A);
    m2.lastB = digitalRead(ENCODER2_B);
}

```

```

attachInterrupt(digitalPinToInterrupt(ENCODER1_A), enc1A, CHANGE);
attachInterrupt(digitalPinToInterrupt(ENCODER1_B), enc1B, CHANGE);
attachInterrupt(digitalPinToInterrupt(ENCODER2_A), enc2A, CHANGE);
attachInterrupt(digitalPinToInterrupt(ENCODER2_B), enc2B, CHANGE);

pidTimer.attach(pidInterval, calcPID);

// PS4 Controller
PS4.attach(ps4Event);
PS4.begin("AC:FD:93:FC:71:22"); //Dirección MAC de mi mando

Serial.println("Esperando conexion PS4...");
}

void loop() {
  if (Serial.available()) {
    long ref = Serial.parseInt();
    if (ref != 0) {
      m1.target = ref;
      m2.target = ref;
      Serial.printf("Nueva referencia: %ld\n", ref);
    }
  }
}

```

Periférico	Implementación	Justificación
PWM	<code>ledcWrite()</code> 4 channels	Precise motor speed control
Interrupciones	4x <code>attachInterrupt(CHANGE)</code>	Real-time encoder odometry
Timers	<code>Ticker</code> FreeRTOS	100Hz multitask PID
Bluetooth	PS4Controller library	Low-latency wireless control

Camera Streaming & Face Detection

The ESP32-CAM runs a web server for live video streaming via WiFi (802.11 bgn). It includes basic image processing for face detection using adapted OpenCV-lite libraries. Allows real-time remote monitoring for rescue or tactical reconnaissance operations.

The camera creates its own WiFi access point so the operator can connect directly and access the stream from any browser, with typical latency of 200–500 ms. It supports JPEG streaming up to UXGA resolution and RGB565 mode for face detection.

MainCam_Program

```
#include "esp_camera.h"
#include <WiFi.h>
#include "board_config.h"

// Camera model selected: AI THINKER
// The camera itself creates a WiFi access point to access the image
const char* ap_ssid      = "ESP32-CAM-AP";
const char* ap_password = "12345678"; // mínimo 8 caracteres

void startCameraServer();
void setupLedFlash();

void setup() {
  Serial.begin(115200);
  Serial.setDebugOutput(true);
  Serial.println();

  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer  = LEDC_TIMER_0;
  config.pin_d0  = Y2_GPIO_NUM;
  config.pin_d1  = Y3_GPIO_NUM;
  config.pin_d2  = Y4_GPIO_NUM;
  config.pin_d3  = Y5_GPIO_NUM;
  config.pin_d4  = Y6_GPIO_NUM;
  config.pin_d5  = Y7_GPIO_NUM;
  config.pin_d6  = Y8_GPIO_NUM;
  config.pin_d7  = Y9_GPIO_NUM;
  config.pin_xclk = XCLK_GPIO_NUM;
  config.pin_pclk = PCLK_GPIO_NUM;
  config.pin_vsync = VSYNC_GPIO_NUM;
  config.pin_href = HREF_GPIO_NUM;
  config.pin_sccb_sda = SIOD_GPIO_NUM;
  config.pin_sccb_scl = SIOC_GPIO_NUM;
  config.pin_pwdn = PWDN_GPIO_NUM;
  config.pin_reset = RESET_GPIO_NUM;
  config.xclk_freq_hz = 20000000;
  config.frame_size = FRAMESIZE_UXGA;
  config.pixel_format = PIXFORMAT_JPEG; //Start streaming
  config.pixel_format = PIXFORMAT_RGB565; //Starts face detection
  config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
```

```

config.fb_location = CAMERA_FB_IN_PSRAM;
config.jpeg_quality = 12;
config.fb_count = 1;

if (config.pixel_format == PIXFORMAT_JPEG) {
    if (psramFound()) {
        config.jpeg_quality = 10;
        config.fb_count = 2;
        config.grab_mode = CAMERA_GRAB_LATEST;
    } else {
        config.frame_size = FRAMESIZE_SVGA;
        config.fb_location = CAMERA_FB_IN_DRAM;
    }
} else {
    config.frame_size = FRAMESIZE_240X240;
#ifdef CONFIG_IDF_TARGET_ESP32S3
    config.fb_count = 2;
#endif
}

#ifdef CAMERA_MODEL_ESP_EYE
    pinMode(13, INPUT_PULLUP);
    pinMode(14, INPUT_PULLUP);
#endif

// Initialize the camera
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err);
    return;
}

sensor_t *s = esp_camera_sensor_get();
// Initialize camera sensors
if (s->id.PID == OV3660_PID) {
    s->set_vflip(s, 1); // flip it back
    s->set_brightness(s, 1); // up the brightness just a bit
    s->set_saturation(s, -2); // lower the saturation
}
// Reduce frame size
if (config.pixel_format == PIXFORMAT_JPEG) {
    s->set_framesize(s, FRAMESIZE_QVGA);
}

#ifdef CAMERA_MODEL_M5STACK_WIDE || defined(CAMERA_MODEL_M5STACK_ESP32CAM)
    s->set_vflip(s, 1);
    s->set_hmirror(s, 1);
#endif

#ifdef CAMERA_MODEL_ESP32S3_EYE
    s->set_vflip(s, 1);
#endif

// Turn on camera LED flash if defined
#ifdef LED_GPIO_NUM
    setupLedFlash();
#endif
WiFi.mode(WIFI_AP);
WiFi.softAP(ap_ssid, ap_password);

```

```

IPAddress IP = WiFi.softAPIP();
Serial.print("AP IP address: ");
Serial.println(IP);    // normalmente 192.168.4.1

startCameraServer();

Serial.println("Camera Ready!");
Serial.println("Conéctate a la WiFi ESP32-CAM-AP y abre http://192.168.4.1");
}

void loop() {
    //Does nothing
    delay(10000);
}

```

Camera Pins

```

// Define the camera and its pins
#if defined(CAMERA_MODEL_AI_THINKER)
#define PWDN_GPIO_NUM    32
#define RESET_GPIO_NUM  -1
#define XCLK_GPIO_NUM    0
#define SIOD_GPIO_NUM    26
#define SIOC_GPIO_NUM    27

#define Y9_GPIO_NUM      35
#define Y8_GPIO_NUM      34
#define Y7_GPIO_NUM      39
#define Y6_GPIO_NUM      36
#define Y5_GPIO_NUM      21
#define Y4_GPIO_NUM      19
#define Y3_GPIO_NUM      18
#define Y2_GPIO_NUM       5
#define VSYNC_GPIO_NUM   25
#define HREF_GPIO_NUM    23
#define PCLK_GPIO_NUM    22

// 4 for flash led or 33 for normal led
#define LED_GPIO_NUM     4

#else
#error "Camera model not selected"
#endif

```

Análisis de Costes

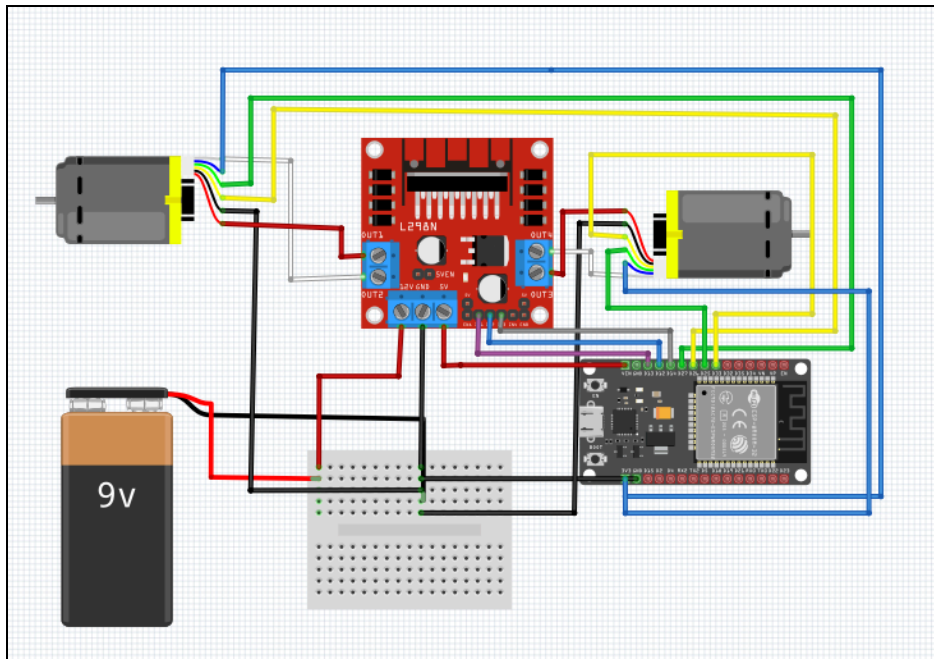
Below is a detailed cost breakdown for manufacturing 10 units of the ground rescue drone, using updated prices. Total cost per unit: €32.85 (hardware) + development costs, competitive against commercial platforms (~€80).

Component	Qty	Unit Price (€)	Subtotal (€)
ESP32-CAM OV2640	1	8,50	8,50
ESP32 Devkit	1	8,00	8,00
L298N Driver	1	9,95	9,95
DC Motor 120 rpm 12V	2	4,50 c/u	9,00
RC Wheels 80mm	2	2,00 c/u	4,00
Wheel Encoders	2	3,56 c/u	7,12
9V NiMH Battery 200mAh	1	5,00	5,00
PLA/PETG Filament (300g)	300g	0,18/g	5,40
Dupont Cables + misc	Lot	-	2,00
9V→5V Regulator	1	1,50	1,50
LEDs + resistors	5	-	1,38
Protoboard/PCB simple	1	-	3,00
Total Hardware/unit	-	-	32,85

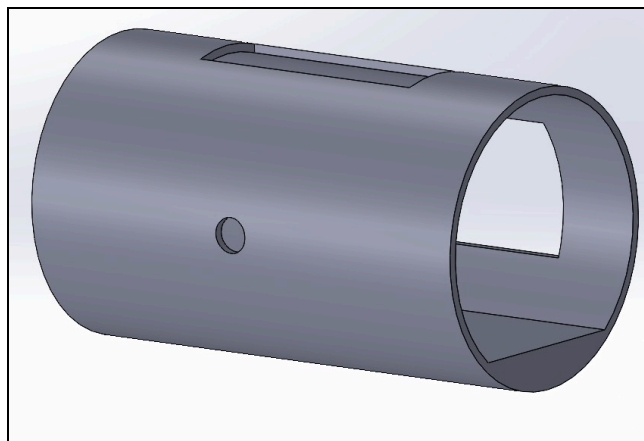
Development Costs (non-recurring)	Hours	Cost (€)
SolidWorks 3D design	15h	225 (15€/h)
Arduino firmware (PS4 + Cam)	20h	300
Testing/prototype	10h	150
3D printing (Ender 3)	8h	40 (electricity)
Total development	53h	715€

Market Comparison	Price(€)	Features vs our drone
mBot Arduino	45	No WiFi camera, no encoders
2WD Platform	25	No PS4 control, no streaming
Ground Rescue Drone	32,85	WiFi live cam + encoders + PS4

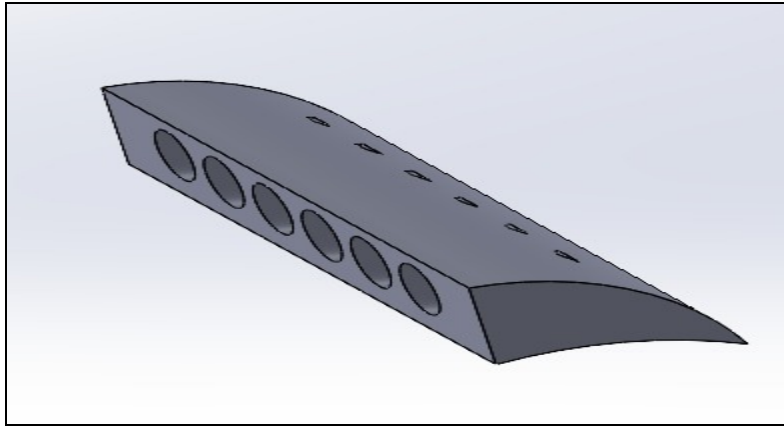
Schematics and Models



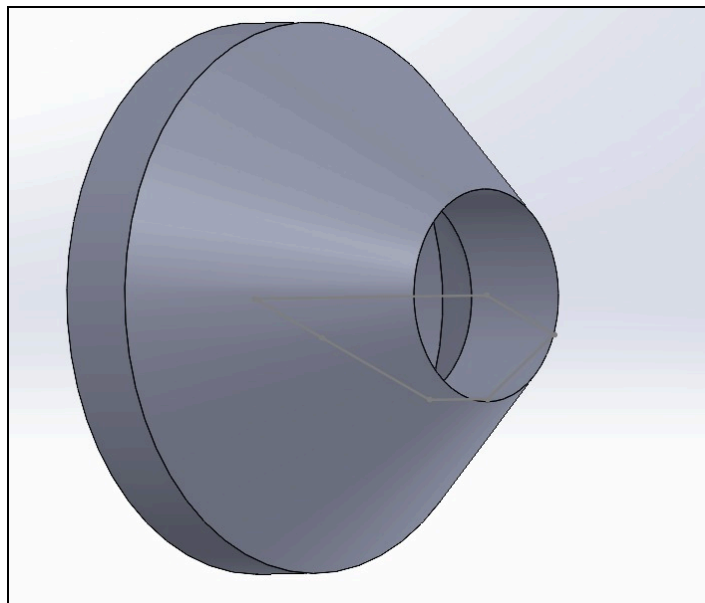
Electrical Wiring Diagram (Fritzing)



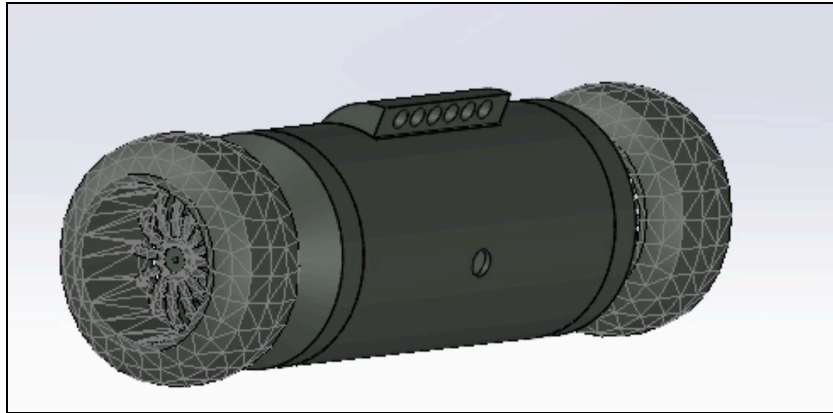
Chassis Design (SolidWorks)



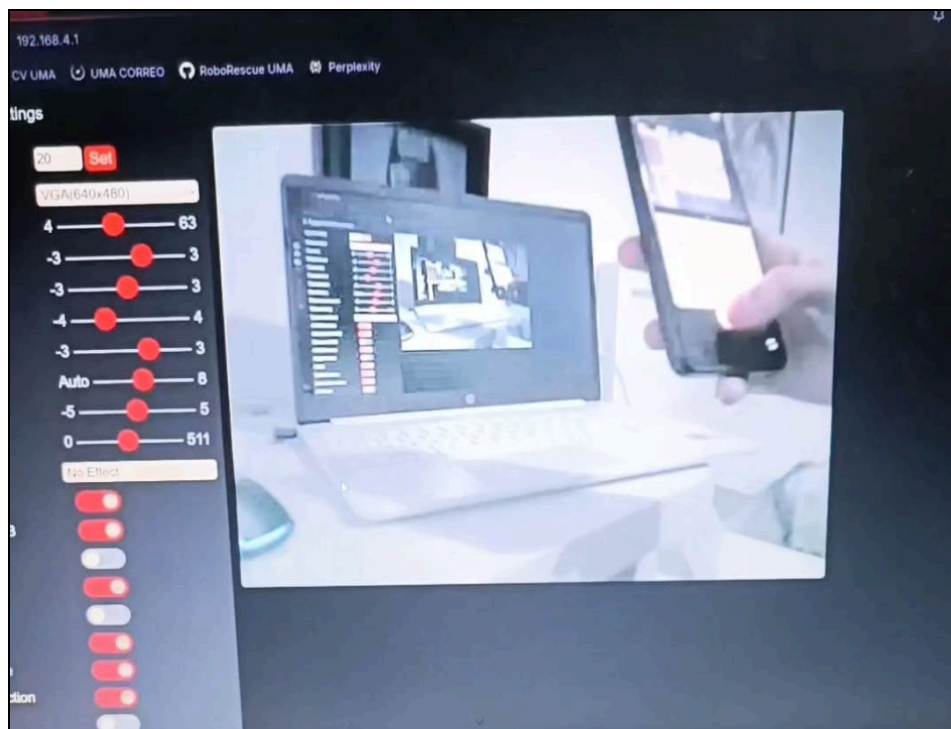
LED Bar Design (SolidWorks)



Motor Mount Design (SolidWorks)



Full Assembly (SolidWorks)



ESP32-CAM Live View